

Machine Learning for Business

GTSC2143

Lecture 9 Clustering I

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

Case: Retrieving documents of interest

Retrieving documents of interest

- Currently reading article you like



Retrieving documents of interest

- Currently reading article you like
- Goal: Want to find similar article



Retrieving documents of interest



Retrieving documents of interest

Challenges

- How do we measure similarity?
- How do we search over articles?



Retrieving documents of interest

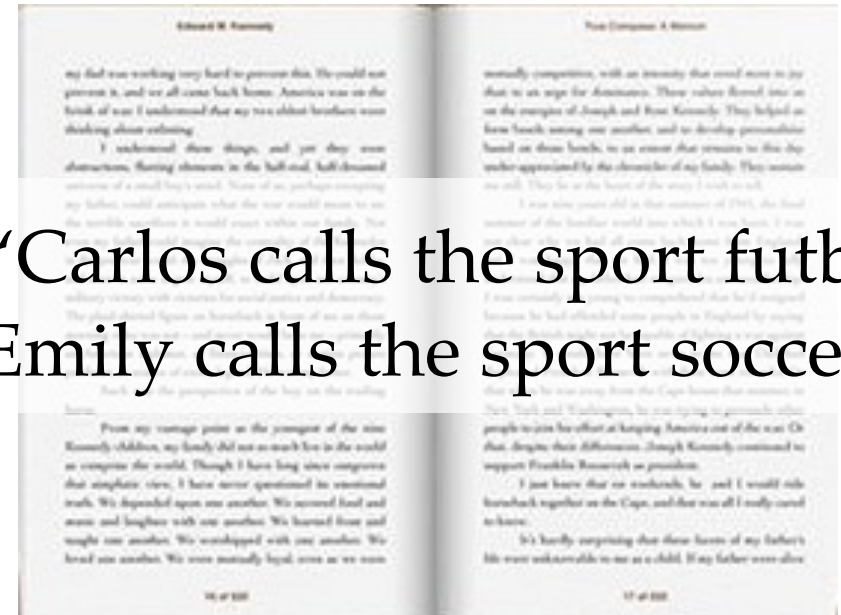
Review: Word count representation

- Bag of words model
- Ignore order of words
- Count # of instances of each word in vocabulary

A simple method we learned in Lecture 7



“Carlos calls the sport futbol.
Emily calls the sport soccer.”



Retrieving documents of interest

Measuring similarity



1	0	0	0	5	3	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---



3	0	0	0	2	0	0	1	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

1×3
 $+ 5 \times 2$
 $= 13$



Retrieving documents of interest

Measuring similarity



Retrieving documents of interest

Issues with word counts – Doc length



Similarity = 13



Similarity = 52



Retrieving documents of interest

Solution = normalize



1	0	0	0	5	3	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

$$\sqrt{(1^2 + 5^2 + 3^2 + 1^2)}$$

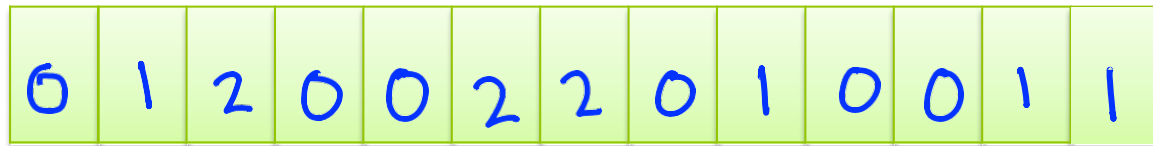
1				5	3			1				
/	0	0	0	/	/	0	0	/	0	0	0	0
6				6	6			6				

Retrieving documents of interest

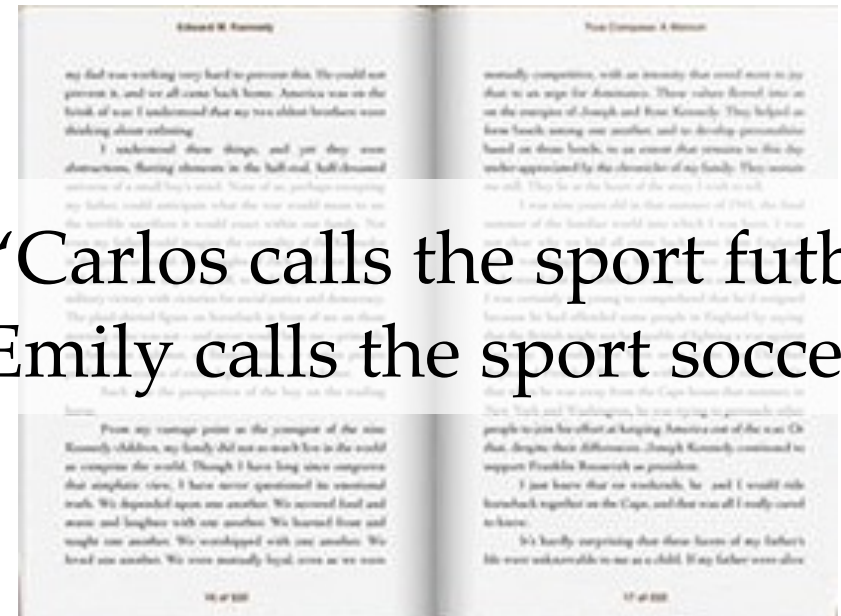
Review: TF-IDF document representation

- Bag of words model
- Ignore order of words
- Count # of instances of each word in vocabulary

A simple method we learned in Lecture 7



“Carlos calls the sport futbol.
Emily calls the sport soccer.”



Retrieving documents of interest

Question

- Is bag of words model suitable enough for retrieving documents?
- If not, which model(s) is/are better?



Retrieving documents of interest

Issues with word counts – Rare words



Common words in doc: “the”, “player”, “field”,
“goal” Dominate rare words like: “futbol”, “Messi”

Retrieving documents of interest

Important words

- Do we want only rare words to dominate???
- What characterizes an important word?
 - Appears frequently in document (common locally)
 - Appears rarely in corpus (rare globally)
- Trade off between local frequency and global rarity

Retrieving documents of interest

TF-IDF document representation

- Term frequency – inverse document frequency (tf-idf)



Retrieving documents of interest

TF-IDF document representation

- Term frequency – inverse document frequency (tf-idf)
- Term frequency



- Same as word counts



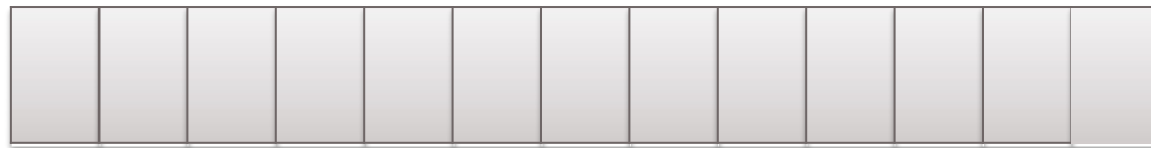
Retrieving documents of interest

TF-IDF document representation

- Term frequency – inverse document frequency (tf-idf)
- Term frequency



- Inverse document frequency



$$\log \frac{\# \text{ docs}}{1 + \# \text{ docs using word}}$$



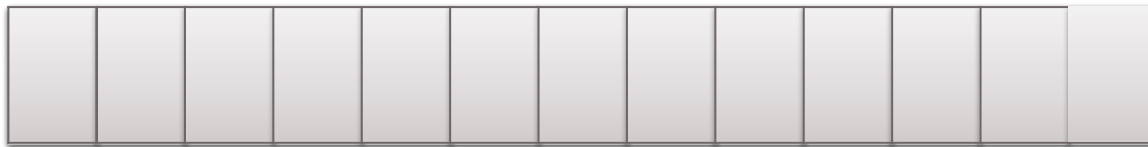
Retrieving documents of interest

TF-IDF document representation

- Term frequency – inverse document frequency (tf-idf)
- Term frequency



- Inverse document frequency



$$\log \frac{\# \text{ docs}}{1 + \# \text{ docs using word}}$$

$\log \frac{\text{large \#}}{1 + \text{large \#}} \approx \log 1 = 0$
word in many docs
 $\log \frac{\text{large \#}}{1 + \text{small \#}} \rightarrow \text{large \#}$
rare word

Retrieving documents of interest

TF-IDF document representation

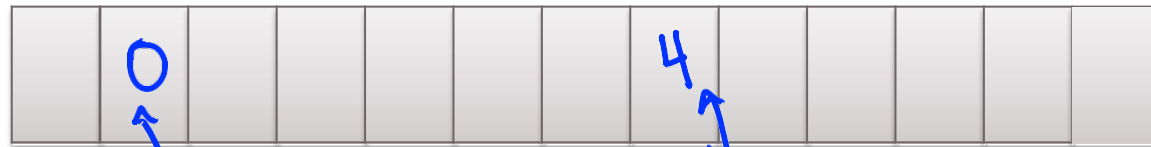
- Term frequency – inverse document frequency (tf-idf)
- Term frequency



the

Messi

- Inverse document frequency

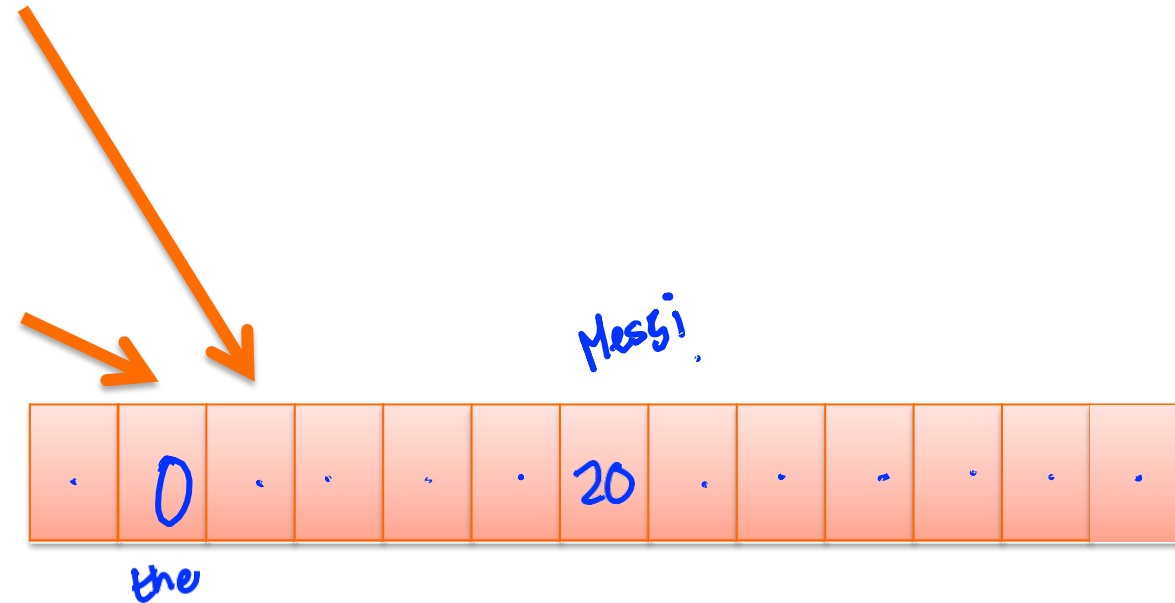


the

Messi

$$\log \frac{64}{1+63} = 0$$

$$\log \frac{64}{1+3} = \log 16$$



Retrieving documents of interest

Discussion

Why is TF-IDF works:

- Emphasizes important words in a document (high TF, low DF).
- Simple and effective for document retrieval.

Limitations:

- Ignores word order and meaning.
- Cannot capture semantic similarity (e.g., “car” $\approx$ “automobile”).

When TF-IDF is not enough:

- For tasks that require understanding meaning or context (e.g., sentiment analysis, question answering).
- In such cases, we might need dense vector representations such as:
 - Word embeddings (Word2Vec, GloVe) — capture semantic similarity.
 - Contextual embeddings (BERT, GPT) — understand meaning from context.

The best feature representation depends on the task!

Retrieving documents of interest

Nearest neighbor search

- Query article:



- Corpus:



- Specify: Distance metric
- Output: Set of most similar articles



Retrieving documents of interest

1 – Nearest neighbor

- Input: Query article 
- Output: *Most* similar article
- Algorithm:
 - Search over each article  in corpus
 - Compute $s = \text{similarity}(\text{query article}, \text{article})$ 
 - If $s > \text{Best_s}$, record  and set $\text{Best_s} = s$ 
 - Return

Retrieving documents of interest

k – Nearest neighbor

- Input: Query article 
- Output: *List of k* similar articles



Clustering documents

Clustering documents

Structure documents by topic

- Discover groups (*clusters*) of related articles



SPORTS

WORLD NEWS

Clustering documents

What if some of the labels are known?

- Training set of labeled docs



SPORTS



WORLD NEWS



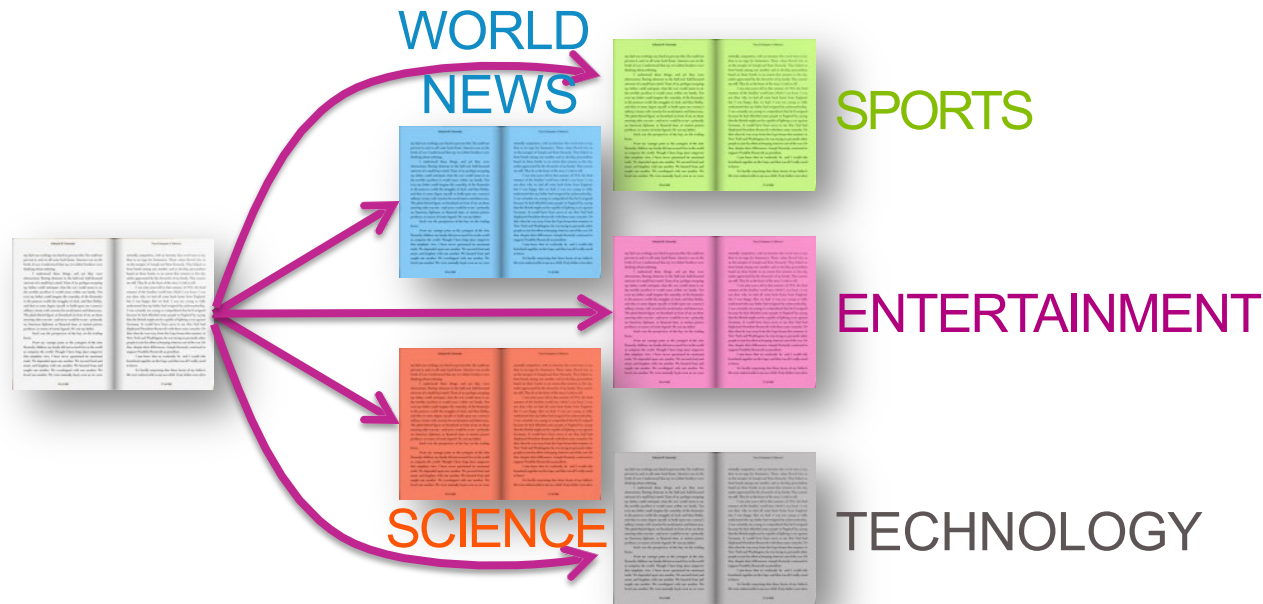
ENTERTAINMENT



SCIENCE

Clustering documents

What if some of the labels are known?



Multiclass classification problem

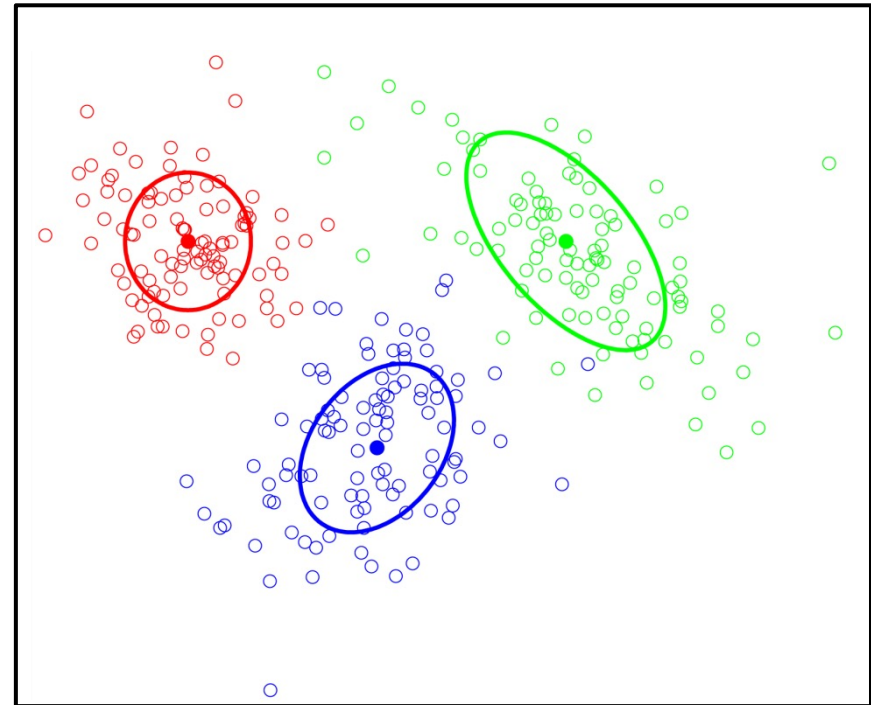
Example of
supervised learning

Clustering documents

Clustering

- No labels provided
- Want to uncover cluster structure
- Input: docs as vectors
- Output: cluster labels

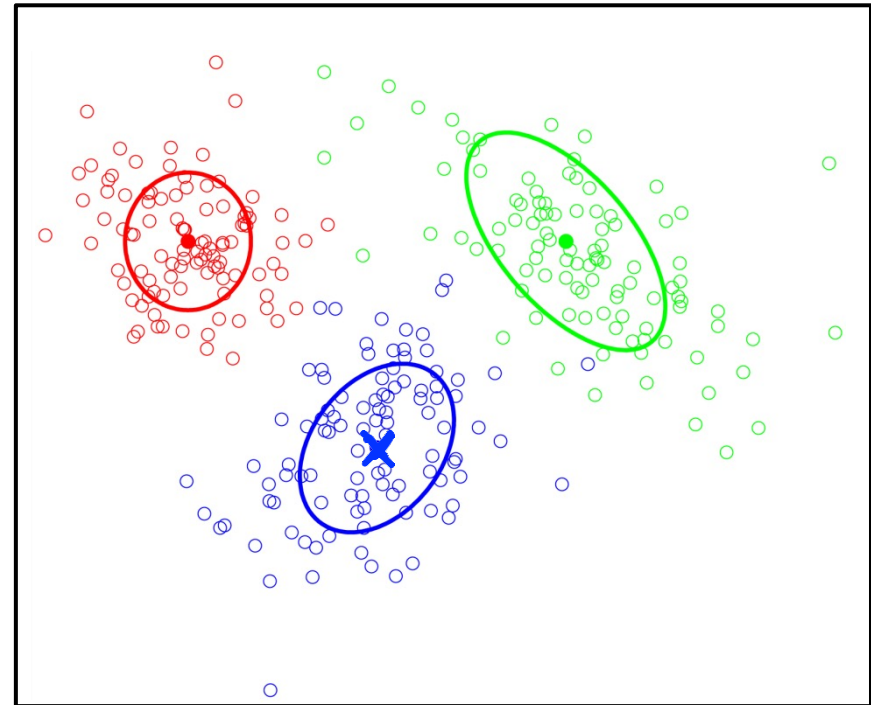
An unsupervised
learning task



Clustering documents

What defines a cluster?

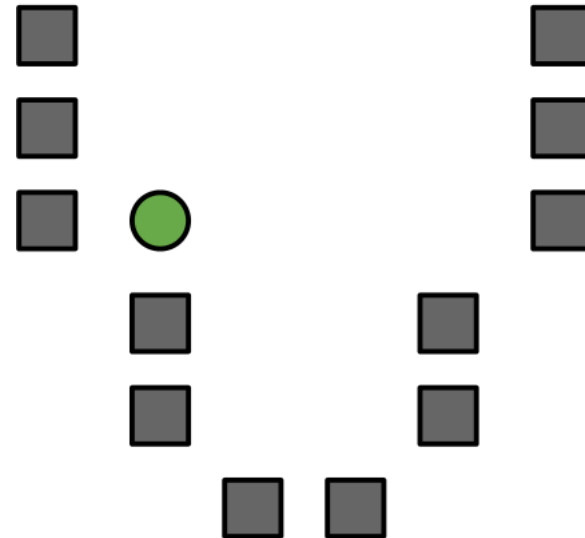
- Cluster defined by center & shape/spread
- Assign observation (**doc**) to cluster (**topic label**)
 - Score under cluster is higher than others
 - Often, just more similar to assigned cluster center than other cluster centers



Clustering documents

k-means

- Assume
 - Similarity metric =
distance to cluster
center
(smaller better)

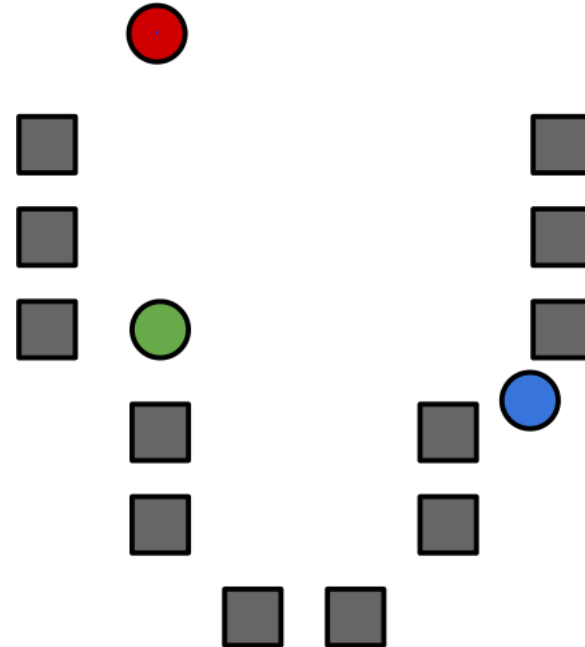


DATA
to
CLUSTER

Clustering documents

k-means algorithm

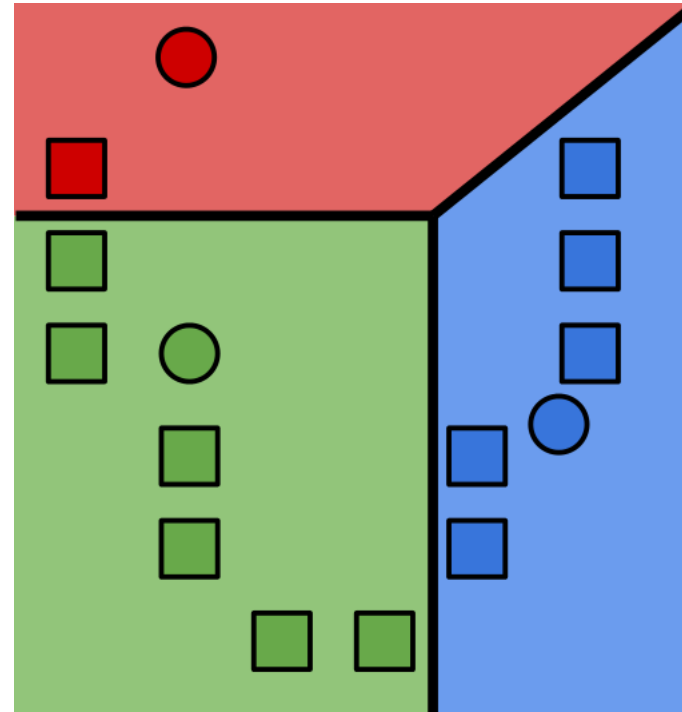
0. Initialize cluster centers



Clustering documents

k-means algorithm

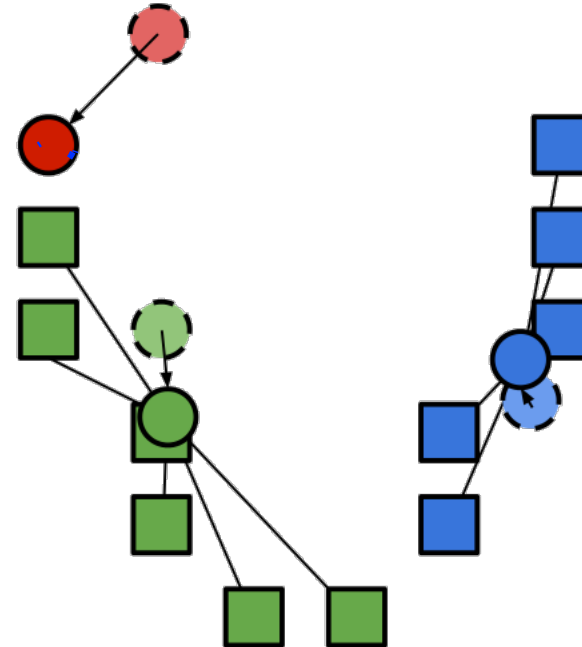
1. Initialize cluster centers
2. Assign observations to closest cluster center



Clustering documents

k-means algorithm

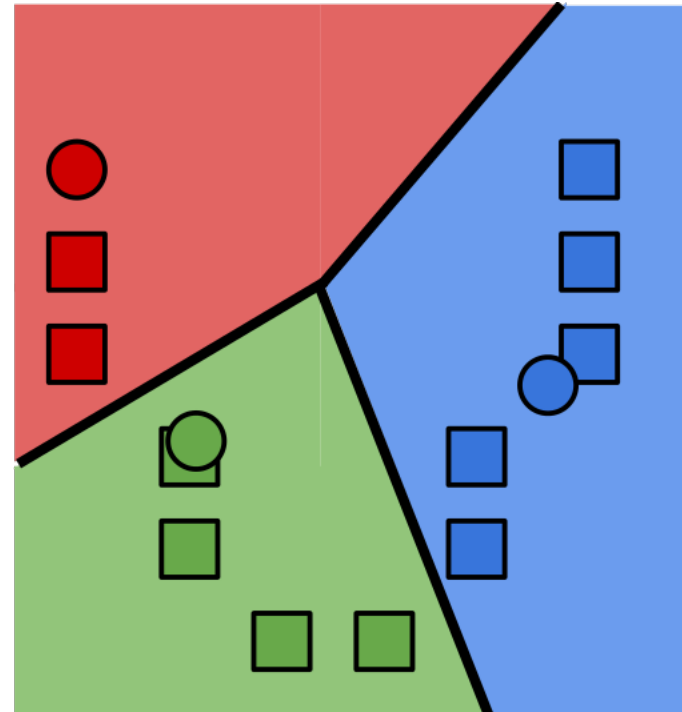
1. Initialize cluster centers
2. Assign observations to closest cluster center
3. Revise cluster centers as mean of assigned observations



Clustering documents

k-means algorithm

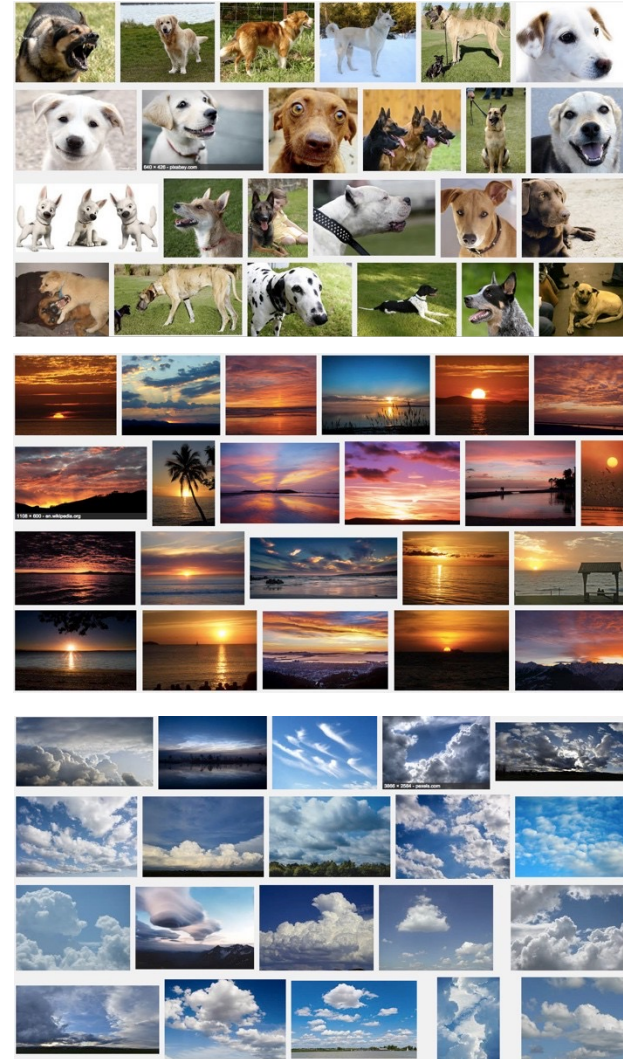
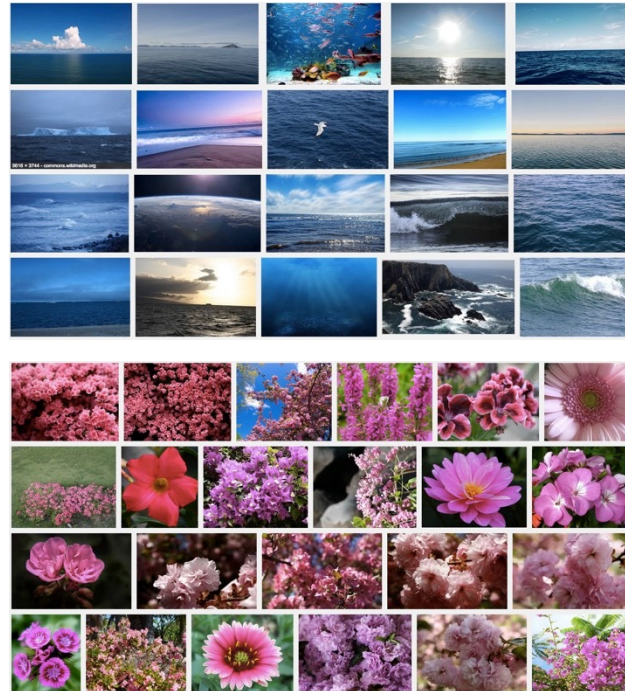
1. Initialize cluster centers
2. Assign observations to closest cluster center
3. Revise cluster centers as mean of assigned observations
4. Repeat 1.+2. until convergence



Other examples of clustering

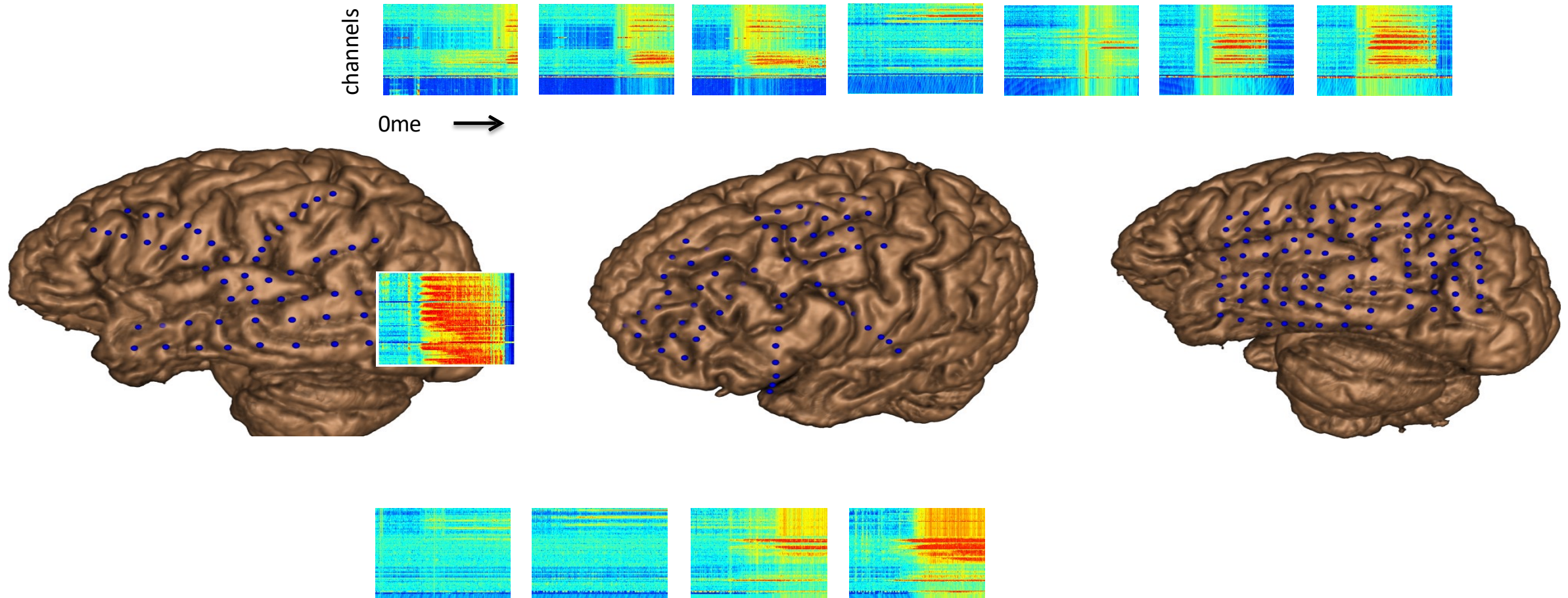
Clustering images

- For search, group as:
 - Ocean
 - Pink flower
 - Dog
 - Sunset
 - Clouds
 - ...



Grouping patients by medical condition

- Better characterize subpopulations and diseases
- Example: Patients and seizures are diverse



Products on Amazon

- Discover product categories from purchase histories



~~"furniture"~~
"baby"



- Or discovering groups of users

Structuring web search results

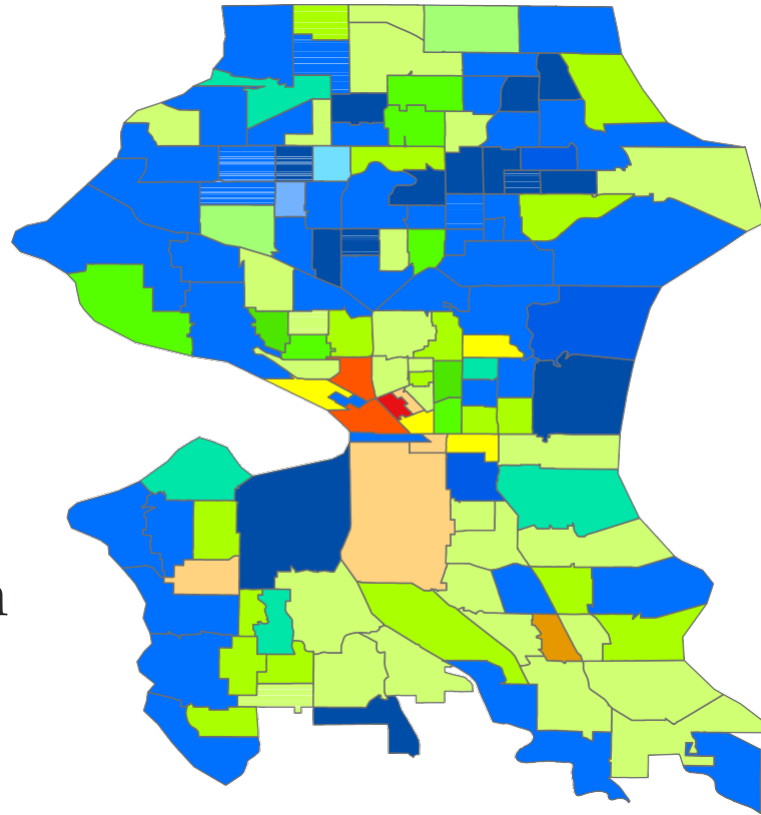
- Search terms can have multiple meanings
- Example: “cardinal”



- Use clustering to structure output

Discovering similar neighborhoods

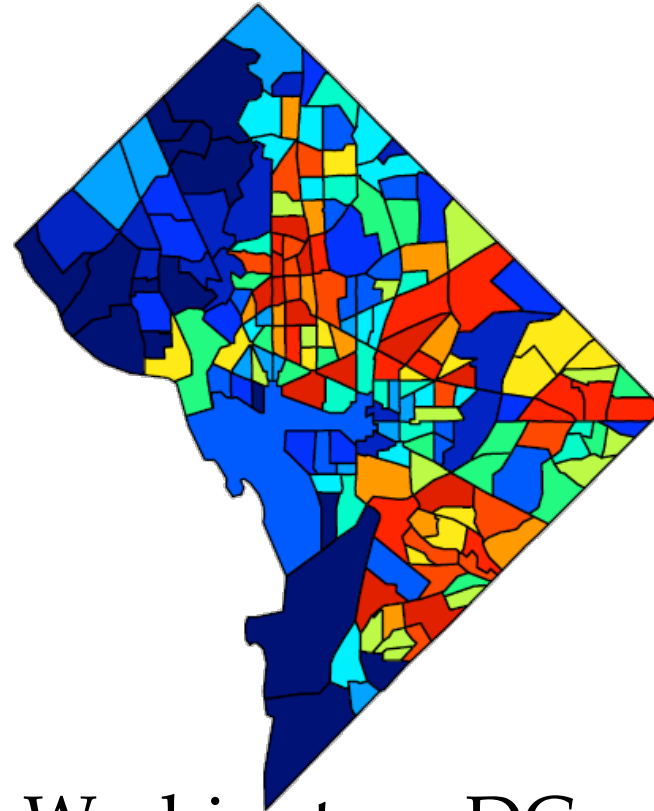
- Task 1: Estimate price at a small regional level
- Challenge:
 - Only a few (or no!) sales in each region per month
- Solution:
 - Cluster regions with similar trends and share information within a cluster



City of Seattle

Discovering similar neighborhoods

- Task 2: Forecast violent crimes to better task police
- Again, **cluster regions** and **share information!**
- Leads to **improved predictions** compared to examining each region independently



Washington, DC

Summary for clustering and similarity

What you can do now...

- Describe ways to represent a document (e.g., raw word counts, tf-idf,...)
- Measure the similarity between two documents
- Discuss issues related to using raw word counts
 - Normalize counts to adjust for document length
 - Emphasize important words using tf-idf
- Implement a nearest neighbor search for document retrieval
- Describe the input (unlabeled observations) and output (labels) of a clustering algorithm
- Determine whether a task is supervised or unsupervised
- Cluster documents using k-means (algorithmic details to come...)
- Describe other applications of clustering

Python Implementation

Python Implementation

TF-IDF representation

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
corpus = [
```

```
    'This is the first document.',
```

```
    'This document is the second document.',
```

```
    'And this is the third one.',
```

```
    'Is this the first document?',
```

```
]
```

- You can set max features to reduce the vocabulary size



```
vectorizer = TfidfVectorizer(
```

```
    max_features=100)
```

```
X = vectorizer.fit_transform(corpus)
```

```
vectorizer.get_feature_names_out()
```

```
>>>array(['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third',  
        'this'], ...)
```

```
print(X.shape)
```

```
>>>[[0. 0.46979139 0.58028582 0.38408524 0. 0. 0.38408524 0. 0.38408524] [0.  
0.6876236 0. 0.28108867 0. 0.53864762 0.28108867 0. 0.28108867] [0.51184851 0.  
0. 0.26710379 0.51184851 0. 0.26710379 0.51184851 0.26710379] [0. 0.46979139  
0.58028582 0.38408524 0. 0. 0.38408524 0. 0.38408524]]
```

Python Implementation

Cosine similarity

```
from sklearn.feature_extraction.text import TfidfVectorizer
corpus = [
    'This is the first document.',
    'This document is the second document.',
    'And this is the third one.',
    'Is this the first document?',
]
vectorizer = TfidfVectorizer(
    max_features=100)
X = vectorizer.fit_transform(corpus)

similarity_matrix = cosine_similarity(X)

print(similarity_matrix)
>>> [[1.  0.64692568  0.30777187  1.  ]
 [0.64692568  1.  0.22523955  0.64692568]
 [0.30777187  0.22523955  1.  0.30777187]
 [1.  0.64692568  0.30777187  1.  ]]
```

Python Implementation

k – Nearest neighbor

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import NearestNeighbors
corpus = [
    "I love machine learning and data science.",
    "Deep learning models are powerful for image recognition.",
    "Natural language processing enables chatbots and translation.",
    "I enjoy learning about data analysis and visualization.",
    "Image classification is a key application of deep learning."
]
vectorizer = TfidfVectorizer(
    max_features=100)
X = vectorizer.fit_transform(corpus)
query = "I am interested in data and machine learning."
query_vec = vectorizer.transform([query])
knn = NearestNeighbors(n_neighbors=3, metric='cosine')
knn.fit(X)
distances, indices = knn.kneighbors(query_vec)
for i, (idx, dist) in enumerate(zip(indices[0], distances[0])):
    print(f"{i+1}. {corpus[idx]}")
    print(f"    Similarity = {1 - dist:.3f}")
```

>>>

1. I love machine learning and data science.
Similarity = 0.7402.
2. I enjoy learning about data analysis and visualization.
Similarity = 0.3923.
3. Natural language processing enables chatbots and translation.
Similarity = 0.114

Python Implementation

k – Means

```
from sklearn.cluster import KMeans
import numpy as np

X = np.array([
    [1, 2],
    [1, 4],
    [1, 0],
    [10, 2],
    [10, 4],
    [10, 0]
])

kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)

print("\nLabels:", kmeans.labels_)
>>>Labels: [1 1 1 0 0 0]
```

Tutorial

